Testing de SW

Que es?

- Proceso destructivo para encontrar defectos en el codigo
- Se debe ir con actitud negativa para demostrar que algo esta mal
- Testing exitoso → el que encuentra algo malo
- Aprox. 30-50% del costo de un sw confiable
- Proceso de corroboracion contra algo.
- No garantiza calidad ni que el producto no tenga defectos. Hace visible los defectos, cuya presencia se asume en el sw
- Testing verifica

Que dice en clases?
Calidad: responsabilidad de todos y cada uno de los q participan en construir el sw. Adaptar prod. o servicios a lo que requiere el cliente. Nivel de cumplimiento del producto o servicio, a las necesidades y expectativas.
Expectativas no son manifiestas, son cosas que el quiere, no importa si las necesita o no, y las evalua.

QA: Conjunto de cosas que hay que hacer **durante** el proceso de construcción del producto. No se puede dejar para el final

Un desarrollo bien hecho, puede llevar a un testing no exitoso (por poco tiempo, no se encuentran defectos → no exitoso)

Valida el cliente, es el unico que puede decir que es lo que quiere.

Asegurar calidad vs. Controlar calidad

- Una vez definidos los req. de calidad, tener en cuenta:
  - Calidad no puede inyectarse al final
  - Calidad depende de las tareas que se hicieron en todo el proceso
  - Detectar errores antes ahorra esfuerzos, tiempo y recursos
  - Calidad abarca tambien la del proceso y como estos se pueden mejorar.
  - Testing NO asegura ni calidad en el software ni software de calidad

Principios del Testing

- Muestra presencia de defecto
- Testing exhaustivo es imposible
- Testing temprano
- Agrupar defectos
- Paradoja del pesticida

- Dependiente del contexto
- Falacia de ausencia de errores
- Programador debe evitar probar su codigo
- Examinar sw. para probar que no hace lo que se supone debe hacer, y que hace lo que no se supone que debe hacer

Cuanto es suficiente?

- Ejemplo:
  - Tengo un sistema con 20 pantallas y 3 menus, 4 opciones, 10 campos por pantalla, con 101 digitos posibles por campo
  - Suponiendo 10 seg por prueba: 20x3x4x10x100 = 240.000 x 10 → 17 semanas
  - Es un monton!!
- Depende de:
  - Evaluar nivel de riesgo
  - Costos asociados al proyecto
- Usamos riesgos para ver:
  - Que testear primero
  - A que dedicarle mas esfuerzo
  - Que no testear por ahora

Criterio de aceptacion

> **ⓘ Que es?**
>
> Es lo que se usa para resolver el problema de determinar cuando una fase de testing se completó

Definidos en terminos de:

- Costos
- % de test corridos sin fallas
- Fallas aun en software
- No hay defectos en una severidad del sw.

Psicologia del testing:
Devs vs. Testers

Conceptos: Error vs Defecto vs Falla

- **Error**: Cuando queda en el contexto de la misma fase de donde se cometió. No pasó a las otras fases
- Cuando sale de esa fase, es un **defecto**.
  En testing se encuentra defectos en el codigo, porque errores no? porque no

escribe código, no puede generar errores (ya que estos son de la misma fase)

- **Falla:** Mal funcionamiento del sistema, que ocurre como consecuencia de un defecto o error.
  - No todos los defectos provocan fallas en el Sw.
  - Ej: defectos de diseños de interfaz de usuario

Conceptos: Defectos, Severidad y Prioridad

- No todos los defectos son iguales, obviamente
- **Severidad:** Cuan grave es el defecto. El equipo de testing asigna severidad.
  1. Bloqueante/Invalidante: No se puede usar el sistema
  2. Critico: No puedo cumplir mi objetivo con esta funcionalidad. Pero si se puede usar.
  3. Mayor: No cumpli objetivo de funcionalidad, pero tengo forma alternativa de hacerlo
  4. Menor: Algo que no me impide el cumplimiento de la funcionalidad.
  5. Cosmetico
- **Prioridad:** Urgencia del negocio
  - La define el usuario final cuando la detecta.
  - Urgencia
  - Alta
  - Media
  - Baja

Niveles de Testing - Prueba

1er Nivel: Testing unitario

- Se prueba cada componente de forma independiente
- Solo componentes individuales
- Se hace con acceso al código bajo pruebas y con el apoyo del entorno de desarrollo ej. framework para pruebas
- Errores se arreglan tan pronto como se encuentran
- Las hace el desarrollador
- Integradas en técnica de TDD
- En el workflow de implementación

2do Nivel: Testing de Integracion

- Para verificar que las partes de un sistema que funcionan bien solas, funcionan bien en conjunto
- Cualquier estrategia debe ser incremental
  - Integracion de arriba hacia abajo
  - Integracion de abajo hacia arriba

- Ideal tener combinacion de ambos esquemas
- Modulos críticos primero
- Puntos clave:
  - Conectar de a poco las partes complejas
  - Minimizar necesidad de programas auxiliares
- Arma un Build.
- Las hacemos en la implementación.
- Con integracion continua, automatizo las pruebas de integración y siempre tengo un Build.

3er nivel: Testing de Sistema

- Se prueba el sistema como un todo
- Para ver si el sistema en su totalidad opera satisfactoriamente
- Entorno de prueba corresponde al de produccion lo mas posible para reducir al minimo el riesgo de incidentes
- Investigar req. funcionales y no funcionales
- Propiedad del workflow de Prueba

4to nivel: Testing de Aceptacion de Usuario

- Realizada por el usuario para ver si la app se ajusta a sus necesidades
- Establecer confianza en el sistema
- Encontrar defectos **no es su foco**
- Comprende desde pruebas en ambiente de laboratorio (alfa) como en ambientes de trabajos reales (beta)
- El usuario hace las pruebas. De esas pruebas depende que nos acepten el producto o no.
- Se hacen en el despliegue

Entorno:
Ambientes para construir software:

- Desarrollo
- Prueba
- Pre-produccion
- Producción

Porque es importante separar ambiente de desarrollo de de prueba?

- Arreglar el "en mi maquina andaba"
- Pruebas de aceptacion en la de desarrollo

Casos de prueba

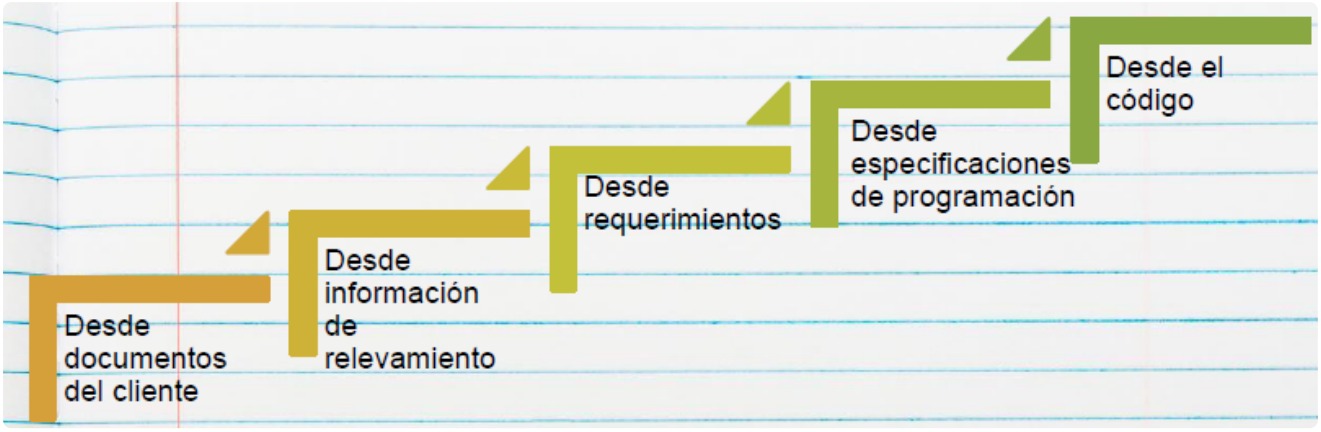
✓ Que son?

- Set de **condiciones o variables** donde un tester determinará si el software esta funcionando correctamente o no
- Unidad de la actividad de la prueba
- Una buena definicion ayuda a **reproducir** defectos
- Los crean los testers
- Solo me sirve para hacer verificacion, no validacion

Tiene 3 partes

- Objetivo: la característica del sistema a comprobar
- Datos de entrada y de ambiente: los que se introducen al sistema, están en las condiciones preestablecidas
- Comportamiento esperado

Generar casos de prueba:



Para diseñar los casos de prueba: sacar desde los requerimientos
Para generar casos

- Ninguna técnica es completa
- Técnicas atacan distintos problemas
- Combinar varias técnicas
- Depende del código a testear
- Sin req. → mas dificil
- Ser sistematico y documentar suposiciones sobre el comportamiento

Condiciones de prueba

✓ Success

Reaccion esperada de un sistema frente a un éstimulo, constituido por distintas entradas.
Deben ser probadas por al menos un caso de prueba

Ciclos de test

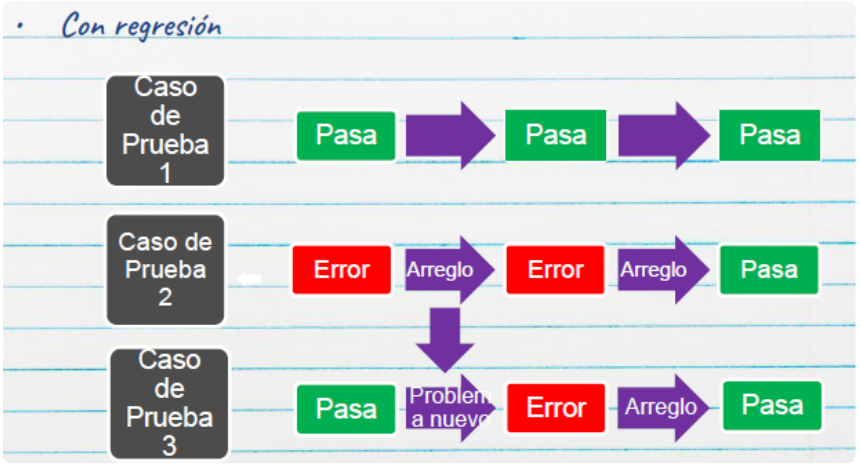
ⓘ Que es?

Ciclo de pruebas abarca la ejecución de todos los casos de prueba establecidos, aplicados a una misma version del sistema a probar

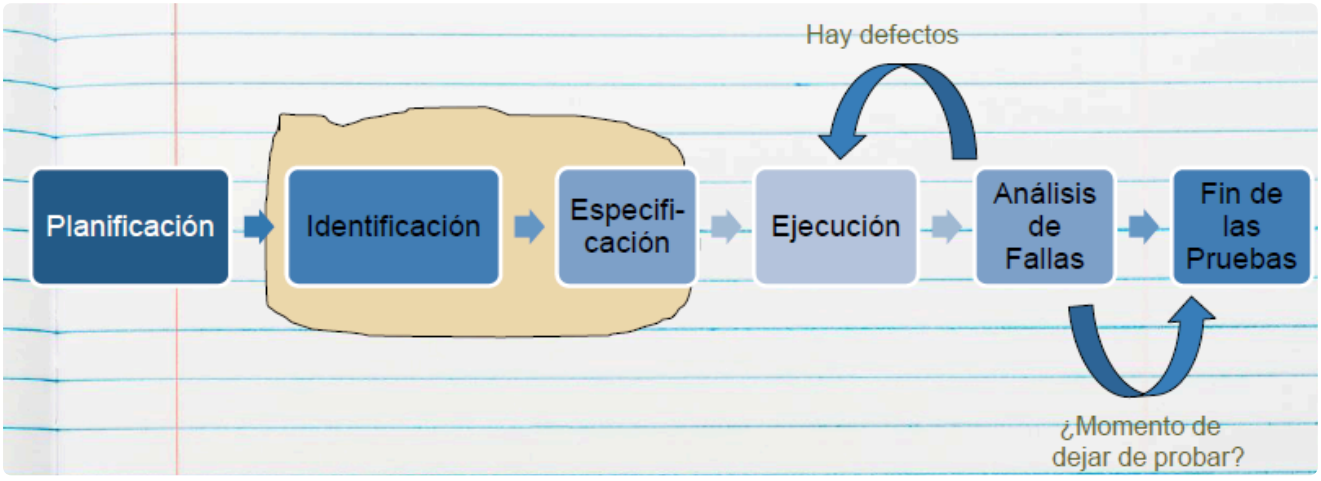
Regresion

ⓘ Que es?

Cuando se concluye un ciclo de pruebas y se reemplaza la version, se debe verificar la nueva versión, con todos los casos de prueba, para prevenir que haya nuevos defectos al querer solucionar otros.



Proceso de pruebas



Planificación de pruebas

- Actividad de verificar que se entiende las metas y objetivos del cliente, los stakeholders, el proyecto, y los riesgos de las pruebas
- Construir el Plan de Pruebas (salida de esta fase) con:
  - Riesgos y objetivos del testing
  - Estrategia de Testing
  - Recursos
  - Criterio de Aceptación
- Controlar:
  - Resultados de testing
  - Test coverage y criterios de acept.
  - Tomar decisiones

Identificación y Especificacion (Diseño)

- Revisar base de pruebas
- Verificar especificaciones para el sw, bajo pruebas
- Evaluar que tan testeables son los requerimientos y el sistema
- Identificar datos necesarios
- Diseñar y priorizar casos de pruebas
- Diseñar entorno de prueba

Ejecución

- Desarrollar y dar prioridad a otros casos
- Crear datos para probar
- Automatizar
- Crear conjuntos de pruebas
- Implementar y verificar el ambiente
- Ejecutar casos de prueba
- Registrar resultado de ejecución de pruebas
- Registrar identidad y versiones del sw
- Comparar resultados reales con esperados
- Resultado: el reporte y las métricas

Evaluar y Reportar

- Evaluar criterios de aceptación
- Reportar resultados de pruebas
- Recolectar info de actividades de prueba completadas
- Verificar entregables y que los defectos se corrigieron
- Evaluar como fueron las actividades de testing

Hint

Verificar: Estamos construyendo el sistema bien?

Validar: Estamos construyendo el sistema correcto?

Ciclo de prueba:
Desde la ejecución al reporte. Si pruebo, reporto la prueba. Vuelvo a ejecutar las pruebas y asi.
Ideal: 2, uno para encontrar y otro para validar defectos.

Estrategias:

Metodos

- Para que usarlos? Tiempo y presupuesto es limitado
- Hay que hacer mayor cantidad de funcionalidad con menor cantidad de pruebas

De Caja negra

- Basado en especificaciones
  - Particion de equivalencias
    1. Dividir el dominio e identificar clases de equivalencia (Validas y no validas) sabiendo que el sistema procesará de la misma forma a todos los elementos que pertenecen a la misma clase
       - Rango de valores continuos
       - Valores discretos
       - Seleccion simple
       - Seleccion multiple
    2. Identificar los casos de prueba
  - Analisis de valores limites
    - Variante de la particion de equivalencias, en vez de elegir cualquier elemento como representativo de una clase de equivalencia, elijo los bordes de una clase.
  - Etc.
- Basados en la experiencia
  - Adivinanza de defectos: Anticipa donde habrá errores comunes con la intuicion del examinador
  - Testing exploratorio:
    - Diseño y ejecucion de pruebas no escritas previamente, guiadas por el conocimiento adquirido mientras se explora el sistema.
- Para hacer el testing, tengo que ejecutar el sw. Son dinámicas

De Caja Blanca

- Basado en el analisis de la estructura interna del sw o un componente de este

- Cobertura de caminos básicos
- Cobertura de sentencias
- Cobertura de decisión
- Cobertura de condición
- Cobertura múltiple
- Etc.
- Son estáticas

desarrollar y acomodar en clases

Elegir un metodo:

- Cada uno tiene sus fortalezas y debilidades
- El mejor metodo es usar varios para un testing efectivo
- Mejor: los dos

Smoke test

> **ⓘ Que es?**
>
> Primer corrida de los test de sistema que asegura que el sw. que esta siendo probado no provoca falla catastrófica.

Tipos de Pruebas

Testing Funcional
Pruebas basadas en funciones y caracteristicas y su interoperabilidad con sistemas especificos

- Basados en requerimientos
- Basados en procesos de negocio

Testing No Funcional

- Prueba de como funciona el sistema
- No hay que olvidarlas
- Son tan importantes como los funcionales

Pruebas de Interfaces de usuario

Pruebas de Performance

- Tiempo de respuesta
- Concurrencia

Prueba de Configuración

- Compatible con X video drivers
- Conexiones de red
- Software de terceros